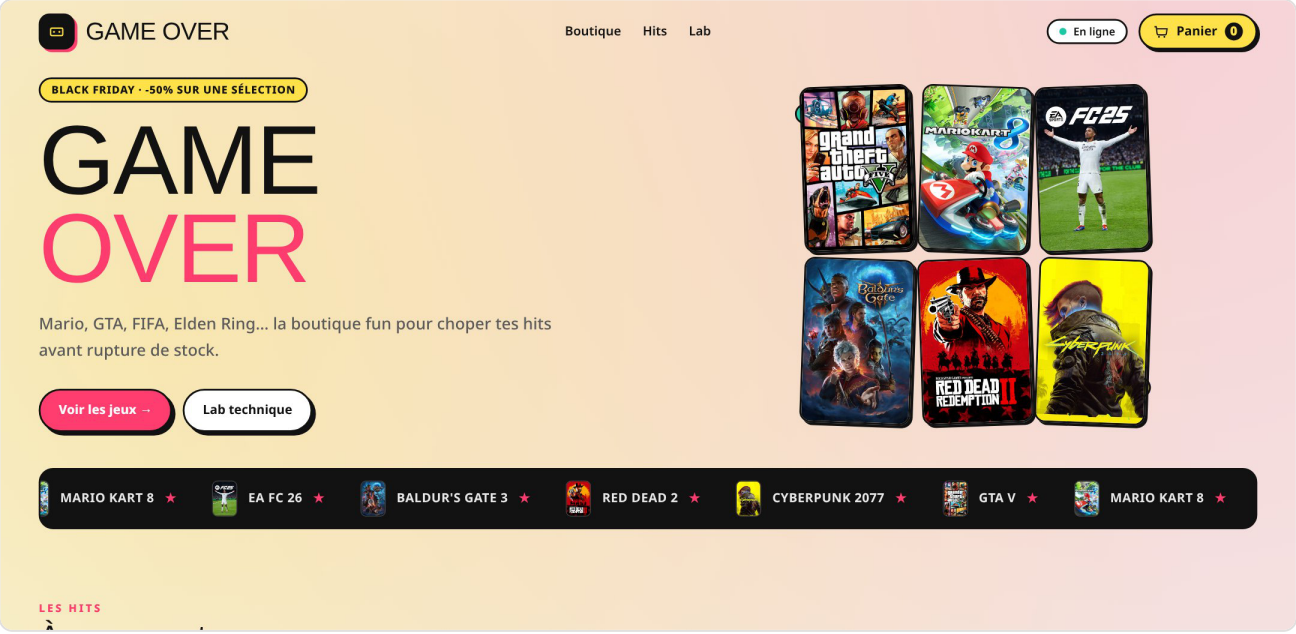
MISSION D'ARCHITECTURE / BOUTIQUE DE JEUX VIDÉO

GAME OVER BLACK FRIDAY

Une boutique prête à ouvrir. Une architecture à remettre d'aplomb avant l'arrivée des clients.



6 missions · **En binôme** · **3 heures environ**
Des problèmes à reproduire, du code à comprendre, des corrections à trouver. Toutes les modifications se font dans `work/`.

COMMENCER ICI

La boutique a besoin de vous

Le Black Friday approche. Game Over vend 12 jeux vidéo, mais plusieurs incidents menacent les ventes : la recherche renvoie de mauvais résultats, des paniers passent avec des données douteuses, le service Stock peut ralentir ou tomber, et le catalogue devient lent sous la charge.

Votre mission : enquêter, comprendre l'architecture et corriger les problèmes. La boutique est déjà développée. Vous n'avez pas à refaire son interface, à créer une base de données ou à remplacer les technologies en place.

- Travail en binôme, **6 missions**, durée indicative : **3 heures**.
- Les missions se font dans l'ordre. La mission 5 comporte une mesure **avant** puis **après** votre correction.
- Vous pouvez explorer le projet et lire `private/`, mais **vos modifications doivent rester dans** `work/`.
- Le rendu final est **le dossier** `work/` **complété**, avec `work/docs/architecture.md`.

Comment utiliser cet énoncé

Pour chaque mission, vous trouverez : **le problème à observer**, **où chercher**, **les comportements attendus**, **une piste d'investigation** et **un test**. Les pistes ne donnent pas l'algorithme : c'est à votre équipe d'identifier la cause et de choisir comment corriger le code.

Les signatures des fonctions exportées et les formats d'échange existants sont des **contrats à conserver**. Vous pouvez ajouter des fonctions internes si vous le souhaitez. Vous n'avez pas à recopier une solution particulière pour valider une mission.

Démarrage : 3 étapes

1. Décompressez le pack, ouvrez `game-over-pld/` dans votre éditeur. Dans un terminal, à côté de `docker-compose.yml`, lancez :

   ```
   docker compose up
   ```

2. Dans votre navigateur, ouvrez **http://localhost:8080**. Vérifiez que vous voyez le catalogue, le panier et, plus bas, la section « **Chaos & perf** ».
3. Gardez ce terminal ouvert. Dans un **second terminal**, toujours à la racine de `game-over-pld/`, lancez :

   ```
   docker compose exec -T api node private/scripts/smoke.mjs
   docker compose exec -T api node private/scripts/progress.mjs
   ```

Le smoke test doit passer. Les tests des missions sont **rouges au départ, volontairement** : ils décrivent ce qu'il vous reste à réparer. **Ne modifiez jamais les tests pour les faire passer.**

Où travailler ?

Dossier ou fichier	Ce que vous pouvez faire
`private/`	**Lire pour comprendre**, jamais modifier. Contient les serveurs, tests, interface et configuration Nginx.
`work/api/`	**Modifier** pour les missions 1, 2, 3 et 5.
`work/web/view-model.mjs`	**Modifier** pour la mission 4.
`work/docs/architecture.md`	**Compléter** avec vos mesures, diagramme et décisions (missions 5 et 6).
`docker-compose.yml`, `README.md`, énoncé et scripts de test	**Ne pas modifier**.

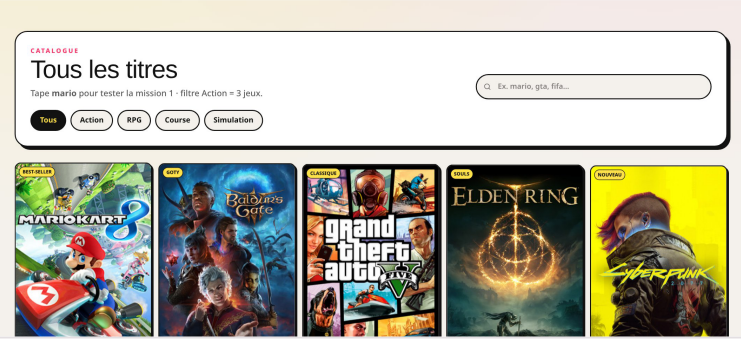
Après un changement dans `work/api/`, redémarrez l'API avec `docker compose restart api`. **Après un changement dans** `work/web/`, actualisez la page. Les modifications se font dans vos fichiers locaux, même si Docker les monte en lecture seule dans les conteneurs.

ENQUÊTE / CORRECTION / VALIDATION

Mission 1 — Une recherche qui ne cherche pas

40 min environ | Architecture en couches

Ce que vous constatez



Dans la boutique : le catalogue et ses filtres.

Dans la boutique, la recherche du catalogue renvoie trop de résultats. Une fiche de jeu inexistante ne renvoie pas non plus la bonne réponse HTTP. En inspectant l'API, vous remarquerez que les responsabilités de certaines couches sont mélangées.

Votre objectif : réparer le catalogue tout en remettant les responsabilités de **Controller**, **Service** et **Repository** au bon endroit, sans créer de nouveaux services Docker.

Où chercher et travailler :

- À **modifier** : `work/api/catalog.controller.mjs` et `work/api/catalog.service.mjs`.
- À **consulter** : `work/api/catalog.repository.mjs` et le chemin de `GET /api/games` dans `private/api/server.mjs`.
- Le Repository est déjà fourni et ne doit pas être modifié.

Comportements attendus

- La recherche par titre accepte une partie du titre ; elle ignore la casse et les espaces en début/fin.
- Le filtre de genre fonctionne avec une correspondance exacte, sans sensibilité à la casse.
- Recherche et genre peuvent se combiner. Un filtre vide ne restreint pas les résultats. Les données source ne sont pas modifiées.
- `GET /api/games` renvoie **HTTP 200**, un tableau `items` filtré et `total` égal au nombre de jeux renvoyés.
- `GET /api/games/:id` renvoie **200** et le jeu s'il existe, sinon **404** avec `{ "error": "GAME_NOT_FOUND" }`.
- Le Controller ne doit plus dépendre directement de la source de données ni du cache. Le Service est responsable des règles du catalogue et de l'utilisation du cache fourni ; le Repository garde l'accès aux données.

À essayer : dans le catalogue, `mario` donne **1 jeu**, le filtre `Action` **3 jeux**, et `mario` + `Action` **0 jeu**. Vous pouvez aussi observer une fiche inexistante :

```
curl -i http://localhost:8080/api/games/introuvable
```

Piste d'investigation : partez d'une requête HTTP et suivez les fichiers appelés. Qu'est-ce qui relève du protocole HTTP, de la recherche métier ou de la lecture des données ? Une couche doit-elle connaître toutes les autres ?

Validation :

```
docker compose restart api
docker compose exec -T api node --test private/tests/01-catalog.test.mjs
```

La mission est terminée quand les **8 tests** passent et que les essais dans la boutique correspondent aux résultats indiqués.

ENQUÊTE / CORRECTION / VALIDATION

Mission 2 — Des commandes trop faciles à tromper

35 min environ | Domaine métier et application

Ce que vous constatez

La boutique accepte des paniers incohérents et peut utiliser un prix envoyé par le navigateur. Une validation faite uniquement dans l'interface ne suffit pas à protéger une commande.

Votre objectif : isoler les règles qui appartiennent au **domaine Commandes** et corriger le traitement d'une commande. Les règles métier doivent rester indépendantes des accès réseau.

Où chercher et travailler : `work/api/order.domain.mjs` et `work/api/order.service.mjs` (à modifier). Pour comprendre les échanges, vous pouvez consulter `POST /api/checkout` dans `private/api/server.mjs`.

Comportements attendus

Situation	Résultat à obtenir
Panier absent, non-tableau ou vide	**400**, erreur `EMPTY_BASKET`
Quantité non entière ou hors de **1 à 5**	**400**, erreur `INVALID_QUANTITY`
Deux lignes pour le même jeu	**400**, erreur `DUPLICATE_GAME`
Jeu inexistant	**404**, erreur `GAME_NOT_FOUND`
Réservation refusée faute de stock	**409**, erreur `OUT_OF_STOCK`
Stock inaccessible	**503**, erreur `INVENTORY_UNAVAILABLE`
Panier valide et stock suffisant	**201**, avec `orderId`, `lines` et `totalCents`

- **Aucun accès au Catalogue ni demande au Stock** pour un panier déjà invalide. Aucune réservation pour un jeu introuvable.
- **Une seule demande de réservation** par commande valide, contenant uniquement les identifiants et quantités, sans prix client.
- Le prix et le titre des lignes confirmées viennent des données serveur. Le total est exprimé en centimes et doit rester fiable, même si le navigateur envoie un faux prix.
- Si la réservation échoue, **pas de confirmation ni d'`orderId`**. Les tableaux reçus ne doivent pas être modifiés.
- Le domaine ne fait **aucun appel HTTP**, ne lit pas le Repository et ne dépend pas du serveur. Le service d'application coordonne les échanges externes.

À essayer : 2 exemplaires de Mario Kart 8 Deluxe doivent coûter **59,80 €**, même avec un faux prix côté client. Testez aussi une quantité `0` et un jeu inconnu. Réinitialisez le stock via « **Chaos & perf** » après vos essais.

Piste d'investigation : quelles vérifications pourraient s'exécuter sans serveur ni connexion réseau ? Quelles opérations nécessitent, au contraire, des données fiables ou une réponse d'un autre service ? Comparez ces responsabilités avec l'organisation actuelle des deux fichiers.

Validation :

```
docker compose restart api
docker compose exec -T api node --test private/tests/02-orders.test.mjs
```

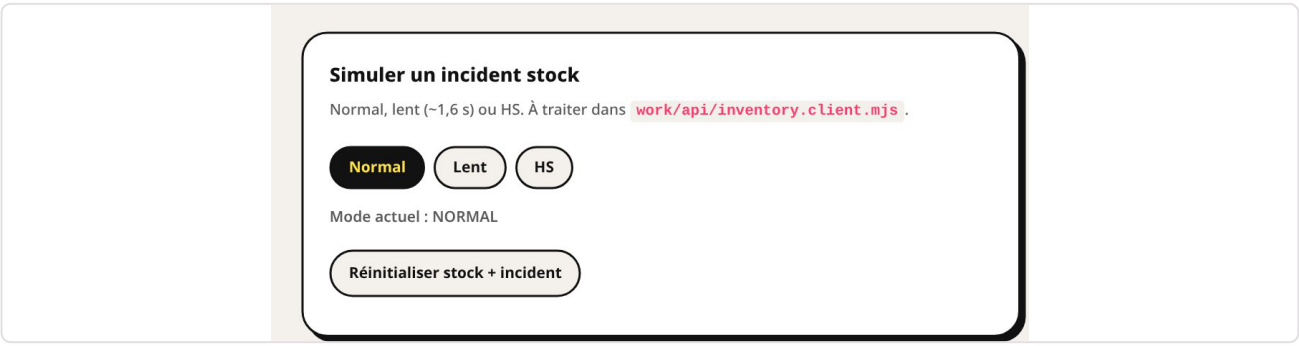
La mission est terminée quand les tests passent et que chaque situation du tableau donne le bon résultat.

ENQUÊTE / CORRECTION / VALIDATION

Mission 3 — Stock en panne, commande en danger

25 min environ | Communication entre services

Ce que vous constatez



Dans la boutique : le simulateur de pannes stock.

Le service Stock fonctionne dans un autre conteneur. Lorsqu'il met trop longtemps à répondre, le traitement des commandes peut rester bloqué. Une panne réseau ne doit jamais devenir une confirmation d'achat.

Votre objectif : rendre fiable l'échange HTTP entre l'API et Stock, y compris lorsqu'une requête doit être abandonnée.

Où chercher et travailler : `work/api/inventory.client.mjs` uniquement. Le service Stock fourni et son simulateur ne sont pas à modifier.

Comportements attendus

- Une demande de réservation reste un **POST** `/reserve` avec un corps JSON `{ "items": [...] }`.
- Une réservation réussie conserve le résultat métier `stock`.
- Un manque de stock conserve une erreur **409** `OUT_OF_STOCK`, avec les informations du jeu lorsque disponibles.
- Si Stock est **HS**, renvoie une erreur serveur, coupe la connexion ou dépasse le délai d'attente (**650 ms par défaut**), le client renvoie **503** `INVENTORY_UNAVAILABLE`. La requête réseau en attente doit pouvoir être **annulée**.
- Les ressources associées à la limite de temps sont libérées, que l'appel réussisse ou échoue.
- **Pas de nouvelle tentative automatique** sur la réservation : une réponse perdue ne prouve pas qu'une opération n'a pas eu lieu.

À essayer dans « Chaos & perf » :

- **Normal** : une commande valide peut être confirmée (**201**).
- **Lent** : Stock attend environ **1,6 s** ; l'API doit répondre **503** avant d'attendre tout ce délai.
- **HS** : une tentative de commande reçoit **503**, jamais une confirmation.
- **Réinitialiser stock + incident** : retour au fonctionnement normal.

Piste d'investigation : provoquez d'abord le mode Lent et observez l'attente. Que se passe-t-il lorsqu'un service distant ne répond jamais ? Et si la réponse se perd après une réservation déjà effectuée ?

Validation :

```
docker compose restart api
docker compose exec -T api node --test private/tests/03-resilience.test.mjs
```

La mission est terminée quand les **4 tests** passent et que les scénarios Normal, Lent et HS sont cohérents.

ENQUÊTE / CORRECTION / VALIDATION

Mission 4 — Un affichage de stock trompeur

20 min environ | Logique de présentation et ViewModel

Ce que vous constatez

Sur la boutique, des jeux épuisés peuvent paraître disponibles. L'interface React existe déjà ; le problème est dans les données préparées pour l'affichage.

Votre objectif : corriger le **ViewModel** de disponibilité, sans toucher aux composants React.

Où chercher et travailler : `work/web/view-model.mjs` uniquement. Conservez les exports et la fonction de formatage des prix déjà fournis.

Comportements attendus

Stock réel	Texte à afficher	État (`kind`)	Achat autorisé (`canBuy`)
0	`Épuisé`	`out`	`false`
1, 2 ou 3	`Plus que N !` (N = quantité)	`low`	`true`
4 ou plus	`En stock`	`ok`	`true`

La fonction de présentation renvoie **un objet de données**. Elle ne manipule ni le DOM ni JSX et ne lance pas d'appel API.

À essayer : après réinitialisation du stock, **Cyberpunk 2077** (0 exemplaire) doit apparaître épuisé, et **Elden Ring** (2 exemplaires) afficher « **Plus que 2 !** ».

Piste d'investigation : quel composant connaît la quantité brute, et où la transforme-t-on en texte, style et disponibilité du bouton ? Ce calcul a-t-il besoin de React ?

Validation :

```
docker compose exec -T api node --test private/tests/04-viewmodel.test.mjs
```

Actualisez la page après modification. Les **6 tests** et les libellés visibles doivent être corrects.

ENQUÊTE / CORRECTION / VALIDATION

Mission 5 — Le catalogue ralentit sous la charge

35 min environ | Cache et scalabilité

Ce que vous constatez



Dans la boutique : le test de montée en charge.

Chaque recherche peut déclencher une lecture coûteuse du catalogue. Sous plusieurs requêtes simultanées, l'API gaspille du temps à effectuer le même travail.

Votre objectif : améliorer les performances avec un cache mémoire fiable, puis comparer les résultats **avant/après**.

Où chercher et travailler : `work/api/cache.mjs`. Complétez vos mesures dans **la rubrique 5** de `work/docs/architecture.md`.

Comportements attendus

- Une valeur déjà calculée pour une clé peut être réutilisée tant que sa **durée de validité (TTL)** n'est pas dépassée. Après expiration, elle est recalculée.
- Des demandes simultanées d'une **même clé** ne déclenchent **qu'un seul calcul partagé** (12 demandes simultanées dans les tests). Des clés différentes restent indépendantes.
- Si un calcul échoue, son échec n'est pas conservé comme une valeur utilisable : un nouvel essai reste possible.
- Une invalidation par préfixe retire les clés concernées, sans effacer les autres. Une invalidation complète doit aussi être possible.
- Un calcul démarré **avant une invalidation** ne peut pas remettre une ancienne valeur dans le cache après celle-ci.
- L'horloge fournie par le contrat existant est utilisable pour vérifier l'expiration ; votre solution doit rester testable sans attendre réellement la fin d'un TTL.

Mesurer dans la boutique

1. **Avant de corriger le cache** : terminez les missions 1 à 4, placez Stock en **Normal**, redémarrez l'API, puis lancez le **test de 48 requêtes** dans « Chaos & perf ». Relevez **p95 en ms** et **requêtes/seconde** dans `work/docs/architecture.md`.
2. Corrigez `work/api/cache.mjs` en respectant les comportements ci-dessus. Redémarrez l'API.
3. Relancez le **même test** dans les mêmes conditions et complétez les valeurs **après**. Expliquez ce que vous observez, même si les chiffres varient selon le poste.

Le simulateur effectue un préchauffage hors mesure. **Aucun seuil de p95 n'est imposé**, mais les quatre valeurs doivent être réellement observées. N'inventez pas de chiffres.

Piste d'investigation : distinguez trois situations : résultat déjà prêt, calcul en cours, résultat devenu périmé. Que doit-il se passer si plusieurs clients arrivent au même moment ?

Validation :

```
docker compose restart api
docker compose exec -T api node --test private/tests/05-cache.test.mjs
```

Les **7 tests** passent et les mesures avant/après sont renseignées avec une explication. Dans votre bilan, répondez aussi à cette question : avec trois instances d'API, que deviennent trois caches locaux ?

ENQUÊTE / CORRECTION / VALIDATION

Mission 6 — Justifier l'architecture de Game Over

25 min environ | Analyse et décisions techniques

Ce que vous constatez

La boutique fonctionne mieux, mais l'équipe doit pouvoir comprendre ce qui a changé et pourquoi l'architecture retenue est adaptée.

Votre objectif : documenter **l'architecture réelle** et justifier des choix pour une éventuelle V2. Vous n'avez pas de nouveau développement à faire.

Où travailler : `work/docs/architecture.md` uniquement (six rubriques à compléter).

Rendu attendu dans ce fichier

1. **Un diagramme Mermaid** montrant le navigateur, l'interface web, l'API, les modules internes et Stock. Les parcours `GET /api/games` et `POST /api/checkout` doivent pouvoir s'y suivre. Distinguez les **couches de code** des **services déployés**.
2. **Une explication des couches et des conteneurs** : qui gère l'affichage, les requêtes, les règles et les données ? Précisez où sont stockées les données dans cette simulation.
3. **Les domaines Catalogue, Commandes et Stock** : une responsabilité et une règle concrète pour chacun. Indiquez qui a autorité sur le prix et qui décide de la disponibilité.
4. **Une comparaison courte MVC / MVVM / MVT**, basée sur ce que vous avez vraiment utilisé, sans réécrire l'application avec trois frameworks.
5. **Vos quatre mesures** de la mission 5, leur interprétation, une limite du cache local et une possibilité de faire évoluer la capacité de l'API.
6. **Une décision argumentée pour la V2** : monolithe modulaire ou microservices pour Catalogue/Commandes ? Stock séparé ou intégré ? Présentez un avantage, un risque et une alternative écartée.

Piste d'investigation : examinez d'abord les conteneurs réellement démarrés, puis les modules utilisés à l'intérieur du processus API. Tout fichier JavaScript n'est pas un microservice. Vous pouvez justifier plusieurs choix d'architecture, du moment qu'ils sont cohérents.

Validation technique :

```
docker compose exec -T api node --test private/tests/06-architecture.test.mjs
```

Le test vérifie surtout la **présence et la structure** du dossier. La qualité du diagramme, des mesures et des justifications sera **relue par le formateur**. Remplacez tous les champs « À compléter ».

DERNIER CONTRÔLE

Vérification finale et rendu

Dans un terminal ouvert dans `game-over-pld/` :

```
# Vérifier que le socle n'a pas été modifié
docker compose exec -T api node private/scripts/check-integrity.mjs

# Vérifier que les services répondent
docker compose exec -T api node private/scripts/smoke.mjs

# Voir les missions validées
docker compose exec -T api node private/scripts/progress.mjs

# Lancer tous les tests
docker compose exec -T api node --test

# Tester le parcours HTTP réel après les missions techniques
docker compose exec -T api node private/scripts/e2e.mjs
```

Dans la boutique, refaites une recherche, un achat valide, une tentative sur un jeu épuisé, un incident Stock, puis une simulation de charge. Réinitialisez le Stock pour finir.

À rendre : le dossier `work/` complet. Vérifiez surtout les fichiers modifiés dans `work/api/`, le ViewModel dans `work/web/` et le document `work/docs/architecture.md`. Aucun PDF supplémentaire, aucune nouvelle interface et aucune base de données ne sont demandés.

En cas de blocage

- **Site inaccessible** : vérifiez `docker compose ps`, puis le terminal de `docker compose up`. Le port utilisé par la boutique est **8080**.
- **Modification API invisible** : lancez `docker compose restart api`, puis consultez `docker compose logs -f api`.
- **Stock épuisé après des essais** : utilisez **« Réinitialiser stock + incident »** dans « Chaos & perf ».
- **Test rouge** : lisez le comportement attendu et l'écart observé ; inspectez les fichiers de **votre mission**. Ne changez pas `private/tests/`.
- **Arrêter le projet** : `Ctrl+C` dans le terminal de Docker, puis `docker compose down` si besoin.

Bonus facultatif : sur votre diagramme seulement, imaginez un CDN et un répartiteur de charge. Expliquez ce qu'ils amélioreraient et ce qu'ils ne résoudraient pas.